Инструкция по работе с Git от 08.09.2025

Ветки

1. master - главная ветка, на ней работает production режим сайта.
2. pre-prod (на тестовых сайтах может быть master) - промежуточная ветка (перед master). Необходима для проверки задач на тесте, и для того, чтобы в случае если постановщик вернул задачу одного разработчика на исправление, другие разработчики могли залить согласованные задачи в ветку master.
3. RS-.... - ветка для разработчика, с номером задачи в Jira.

Важные запреты:

1. **На боевом сайте в ветке master ЗАПРЕЩАЕТСЯ править код!** Даже в критических ситуациях, сначала правим на своей ветке, затем мерджим ее с master и заливаем на боевой сайт.
2. Никогда не форсируйте пуш (--force) в общую ветку master.
3. Не мерджите конфликтующие изменения без проверки.
4. В репозитории Git и на боевом сайте должна существовать только одна ветка - master (если конечно у клиента нет других веток).

Процесс работы в dev среде (локальный компьютер)

Шаг 1: Прежде всего нужно забрать изменения с ветки master, чтобы у вас было актуальное состояние кода. Для этого вводим команды:

git checkout master
git pull origin master

Шаг 2: После того как изменения из репозитория обновились, создавайте новую ветку с номером задачи в Jira (в формате: RS-номер задачи, например RS-2344):

git checkout -b RS-номер задачи

Шаг 3: В ветке с номером задачи, вы выполняете свою задачу.

Шаг 4: После окончания выполнения, добавляете в индекс и делаете коммит с осмысленным сообщением, что было сделано в задаче. Коротко и понятно:

git add .

git commit -m “Текст коммита, что было сделано”

Шаг 5: Пушите изменения своего локального компьютера в удаленный репозиторий:

git push origin RS-номер задачи

Процесс работы в pre-production среде (тестовый сервер)

Шаг 6: Предварительно проверяем, не вносил ли кто изменений на тестовом сайте и в удаленный репозиторий. Для этого смотрим реестр коммитов в системе контроля версий Bitbucket, а также выполняем команду в SSH поочередно для веток master и pre-prod (на тестовых сайтах может называться master):

git status

Шаг 7: По факту выполнения смотрим:

- *если ветка master чистая, то переходим к шагу 8*
- *если на ветке есть незакоммиченные изменения, выясняем, кто вносил изменения, для какой задачи, создаем коммит. Переходим к шагу 8.*
- *если вместо ветки master включена ветка с номером задачи, то выясняем, кто вносил изменения, завершены ли работы и спрашиваем, можно ли переключиться на master. Переходим к шагу 8.*

Шаг 8: На тестовом сервере создаем ветку с номером задачи Jira, которую вы ранее сделали и задача готова к демонстрации постановщику:

git checkout -b RS-номер задачи

Шаг 9: Забираете изменения из удаленного репозитория в ветку с номером задачи в Jira:

git pull origin RS-номер задачи

Шаг 10: Переключаетесь на ветку pre-prod (на тестовых сайтах может называться master):

git checkout pre-prod или master

Шаг 11: Оставаясь на ветке pre-prod (на тестовых сайтах может называться master), мерджите ветку с номером задачи с веткой pre-prod через:

git merge RS-номер задачи

Дождитесь, когда постановщик проверит вашу задачу.
Если постановщик вернул задачу на доработку, то вернитесь на Шаг 3 и доработайте задачу. После завершения, повторите шаги 4-11.
Если постановщик утвердил задачу, выполняйте шаг 12, ниже.

Шаг 12: Переключаетесь на ветку master:

git checkout master

Шаг 13: Оставаясь на ветке master, мерджите ветку с номером задачи с веткой master через:

git merge RS-номер задачи

Если возникают конфликты, разрешаете их вручную. Если непонятно, как их разрешить, пишите в общий чат разработчиков, уточняйте.

Шаг 14: После проверки на тестовом сайте, пушите изменения в удаленный репозиторий:

git push origin master

Процесс работы в production среде

Заходите на боевой сайт по SSH, открываем терминал и скачиваем изменения из репозитория:

git pull origin master